Object-Oriented Programming

Further Programming

RMIT UNIVERSITY

Learning Outcomes



Describe basic components of Java classes and objects, along with relationships between classes

Explain the difference between method overloading and method overriding and **apply** in Java code

Describe the purpose of the `java.lang.Object` class and **apply** in Java code

Describe the purpose of Inheritance and **apply** in Java code

Describe the behaviour of attributes, methods and visibility in a Java inheritance hierarchy and **apply** in Java code

Describe inheritance related Java keywords such as `extends`, `final`, `this` and `super` and **apply** in Java code

Describe the purpose of Polymorphism and **apply** in Java code

Describe the intrinsic relationship between Inheritance and Polymorphism

Describe the mechanics of constructors in an inheritance hierarchy and **apply** in Java code

Learning Outcomes



Describe the purpose and application of abstract classes and **demonstrate** how the `abstract` keyword is used in Java code

Demonstrate the use of abstract classes for simple code reuse (i.e. no polymorphism)

Describe and **demonstrate** in code the use of abstract classes for polymorphism

Describe and **demonstrate** in code the use of Java interfaces for polymorphism

Explain the difference between abstract classes and interfaces and the advantages/disadvantages of each approach

Describe the limitations of multiple inheritance in Java and **demonstrate** in Java code how interfaces can be used to overcome these limitations

Explain static versus dynamic types and demonstrate how to choose appropriate types in a class hierarchy when declaring variables and parameters
Demonstrate in code how to avoid the use of `instanceof` by careful use of OO/Java typing
Describe and **demonstrate** in code, delegation as an alternative to inheritance

Classes vs Objects

RMIT
UNIVERSITY

Classes vs Objects

- Classes and objects are the core concepts in OOP
- Examples:

Class	Objects
Fruit	Apple Banana Mango

Class	Objects
Car	Toyota Volvo Audi

- A class is a template for objects, and an object is an instance of a class
- An object inherits the attributes and methods from the class that is used to create the object

Attributes

- Attributes are the data of a class
- They are variables defined in a class

```
// File: Point.java
public class Point {
    double x;
    double y;
}
```

Attributes

- Create objects and access attributes

```
// File: Main.java
public class Main {
    public static void main(String[] args) {
        Point p1 = new Point();  // create a new object
        Point p2 = new Point();
        p1.x = 1; p1.y = 2;
        p2.x = 3; p2.y = 4;
        System.out.println("[" + p1.x + ", " + p1.y + "]");
        System.out.println("[" + p2.x + ", " + p2.y + "]");
    }
}
```

Methods

- A method is a block of code that is used to perform a specific task
- Methods are defined in a class

```
// File: Point.java
public class Point {
    double x, y;
    void showPoint() {
        System.out.println("[" + x + ", " + y + "]");
    }
}
```

Methods

- Methods can be called as follow:

```
// File: Main.java
public class Main {
    public static void main(String[] args) {
        Point p1 = new Point();
        Point p2 = new Point();
        p1.x = 1; p1.y = 2;
        p2.x = 3; p2.y = 4;
        p1.showPoint();
        p2.showPoint();
    }
}
```

Constructors

- A constructor is a **special method** that is used to initialize the values of object attributes
- **this** is a **reference** to the current object

```
public class Point {
    double x, y;
    public Point(double x, double y) {
        this.x = x;          // this.x is the attribute x
        this.y = y           // where x is the local variable x
    }
    void showPoint() {
        System.out.println("[" + x + ", " + y + "]");
    }
}
```

Constructors

- Now we can create objects with the constructor :

```
// File: Main.java
public class Main {
    public static void main(String[] args) {
        Point p1 = new Point(1, 2);
        Point p2 = new Point(3, 4);
        p1.showPoint();
        p2.showPoint();
    }
}
```

Modifiers

- There are two group of modifiers
  - Access modifiers
  - Non-access modifiers

Access Modifiers

- For classes, you can use either **public** or *default* (no modifier):

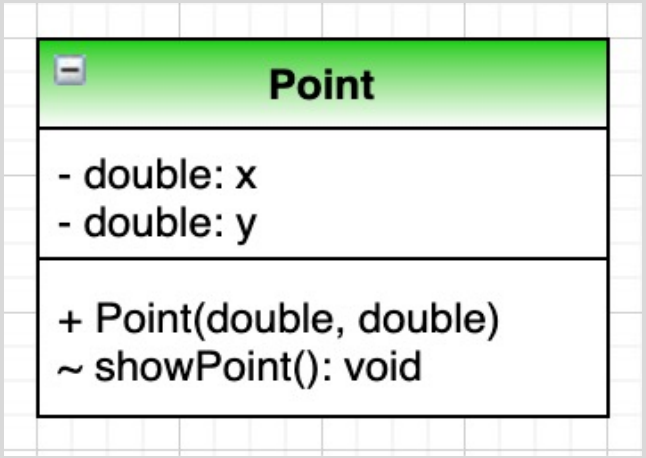
Modifier	Description
public	The class is accessible by any other class
default	The class is only accessible by classes in the same package

- For attributes, methods and constructors, you can use:

Modifier	Description
public	The code is accessible for all classes
protected	The code is accessible by subclasses, or classes in the same package
default	The code is only accessible by classes in the same package
private	The code is only accessible within the class it is declared

Access Modifiers

- Let's apply access modifiers now

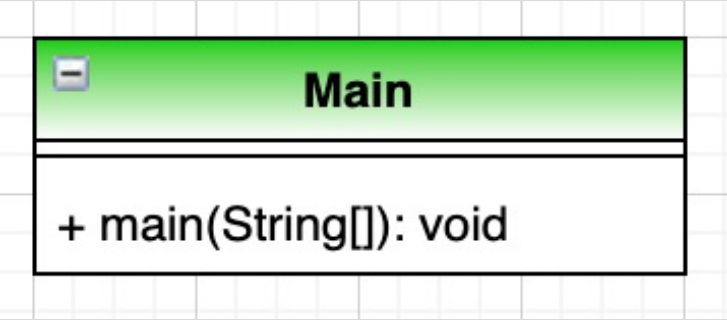



```
// File: Point.java
public class Point {
    private double x, y;
    public Point(int x, int y) {
        this.x = x;
        this.y = y
    }
    void showPoint() {          // default access modifier
        System.out.println("[" + x + ", " + y + "]");
    }
}
```

Access Modifiers

- Now we can create objects with the constructor :

```
// File: Main.java in the same directory with Point.java
public class Main {
    public static void main(String[] args) {
        Point p = new Point(1, 2);
        p.showPoint();
        System.out.println(p.x + ", " + p.y); // error
    }
}
```



Non-Access Modifiers

- For classes, you can use either **final** or **abstract**:

Modifier	Description
final	The class cannot be inherited by other classes
abstract	The class cannot be used to create objects

- For attributes, methods and constructors, you can use one of the following:

Modifier	Description
final	Attributes and methods cannot be overridden/modified
static	Attributes and methods belongs to the class, rather than an object
abstract	Can only be used on methods in an abstract class
transient	Attributes and methods are skipped when serializing the object containing them
synchronized	Methods can only be accessed by one thread at a time
volatile	The value of an attribute is not cached thread-locally, and is always read from the "main memory"

Encapsulation

- Encapsulation is to make sure that the "sensitive" data of a class is hidden from other classes
- To achieve this, you must:
  - Declare the attributes as **private**
  - Provide public **get** and **set** methods, *when needed only*, to access and update the value of a **private** attribute

Packages

- Package is a mechanism to group related classes together
- You can think of a package as a **directory** containing related classes
- Packages allow to structure better maintainable code and to avoid name conflicts
- There are two group of packages:
  - Built-in packages (from the Java API)
  - User-defined packages (your own-created packages)

Packages

- To use a class or a package, use the **import** keyword

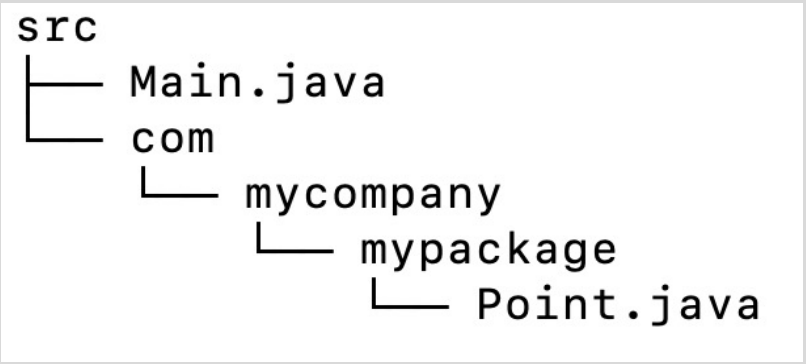
```
import package.name.ClassName;
import package.name.*;

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter your name");
        String name = scan.nextLine();
        System.out.println("Hello " + name);
    }
}
```

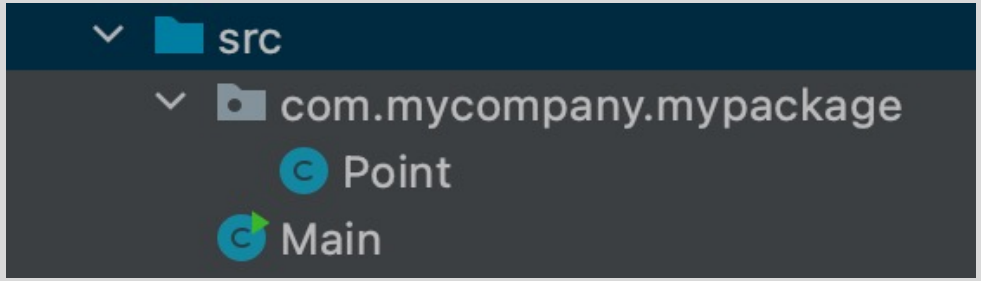
Packages

- Java uses a directory to store the Java files in a package
- To create a package, use the **package** keyword

```
package com.mycompany.mypackage;
public class Point {
    private double x, y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public void showPoint() {
        System.out.println("[" + x + ", " + y + "]");
    }
}
```



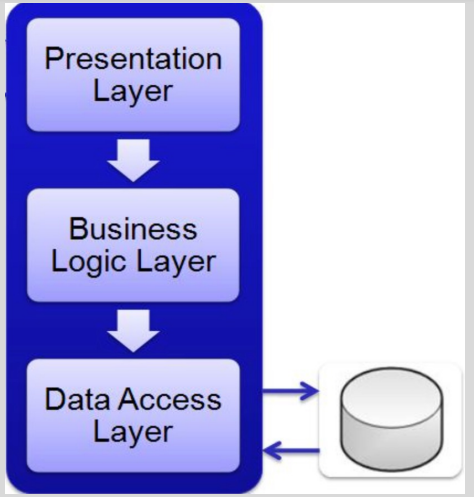
Packages

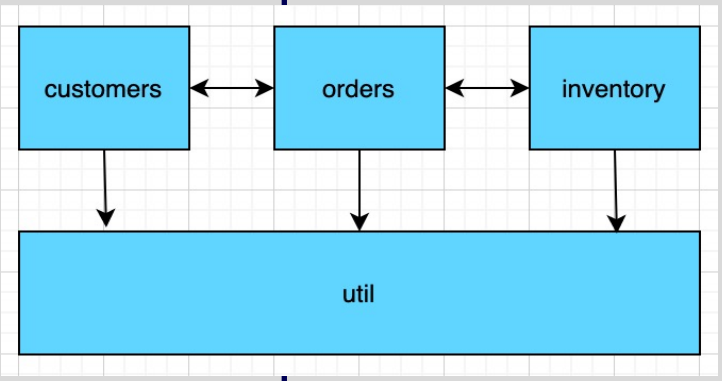



```
import com.mycompany.mypackage.Point;
public class Main {
    public static void main(String[] args) {
        Point p1 = new Point(1, 2);
        Point p2 = new Point(3, 4);
        p1.showPoint();
        p2.showPoint();
    }
}
```

Packages

- Two common approaches to organize your classes into packages
  - By layers: organize your classes based on the app layers
    - Example: presentation, business logic, data access
  - By functionality: organize your classes based on the app functionalities
    - Example: customers, orders, inventory, util
- You can also combine these two approaches



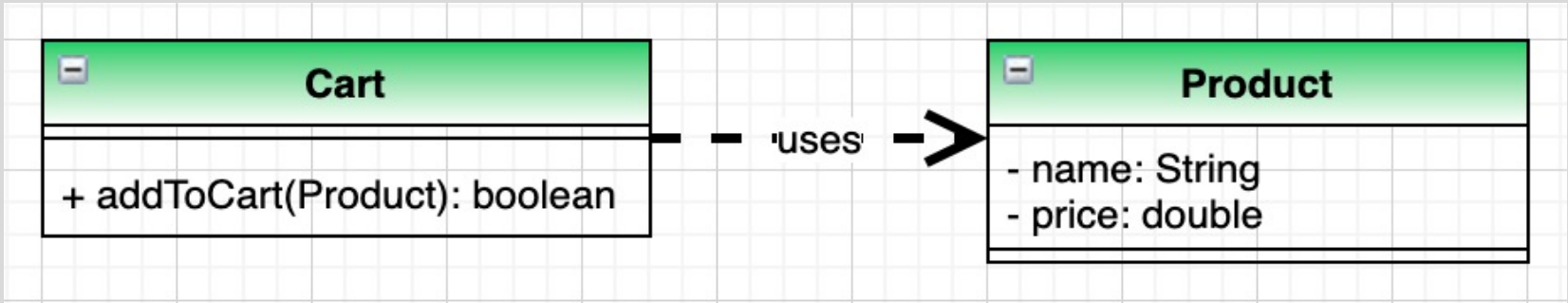


Class Relationships

- To construct a program, classes are designed to relate to each other in one of the following relationships:
  1. Dependency
  2. Association
  3. Aggregation
  4. Composition
  5. Inheritance (Generalization)
  6. Implementation (Realization)

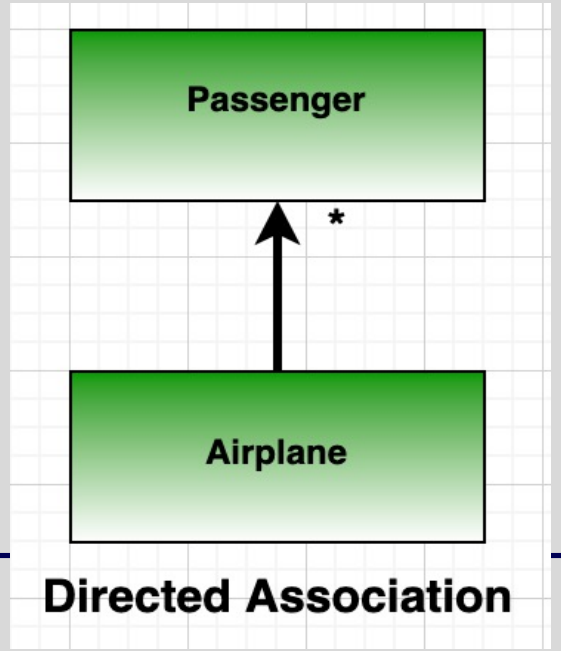
Dependency

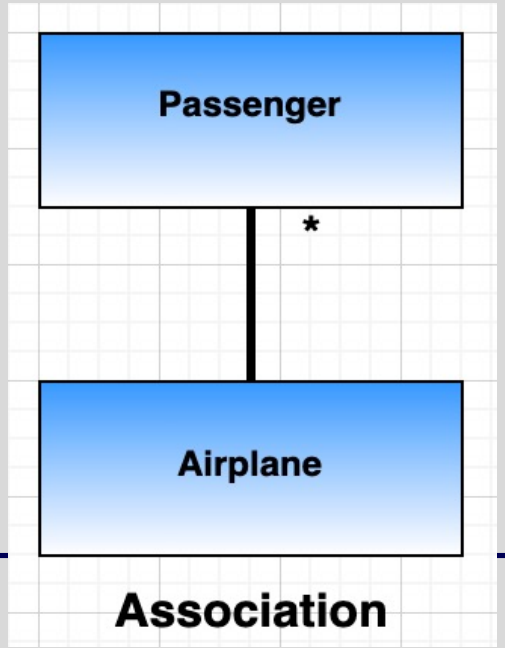
- Dependency represents a "uses" relationship in which a client class "uses" a supplier class
- Dependency indicates that the client class may:
  - use the supplier class as a parameter / local variable / return value in one or more methods
  - call one or more methods of the supplier class



Association

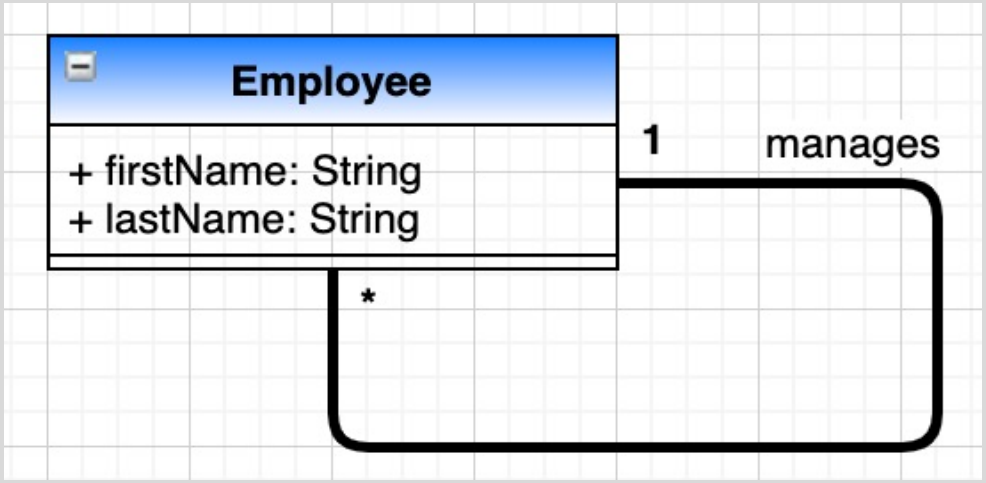
- Association can represent any relationship between two classes
- Association indicates that a class may:
  - use the other class as one or more attributes
  - call one or more methods of the other class





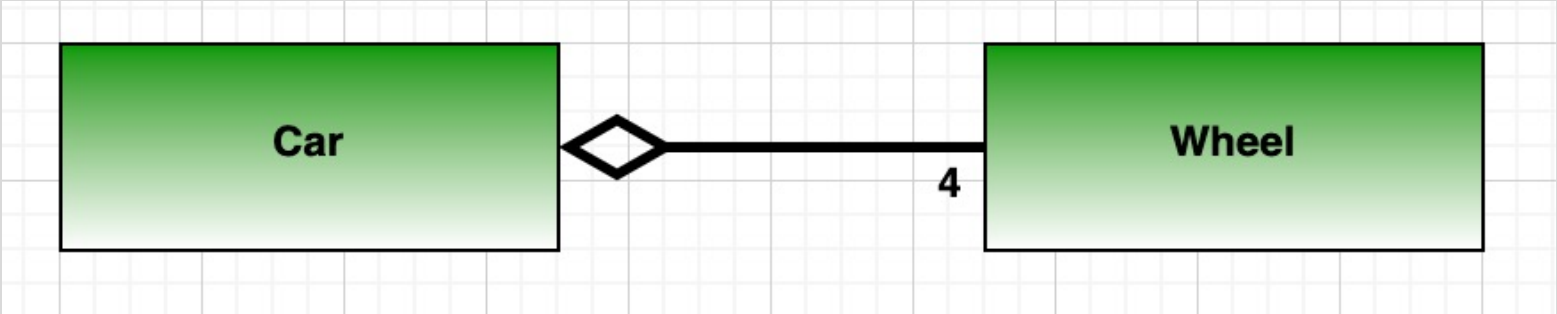
Reflexive Association

- Reflexive association is used when a class is related to itself
- In this example, an Employee object has a role Manager thus manages many other Employee objects



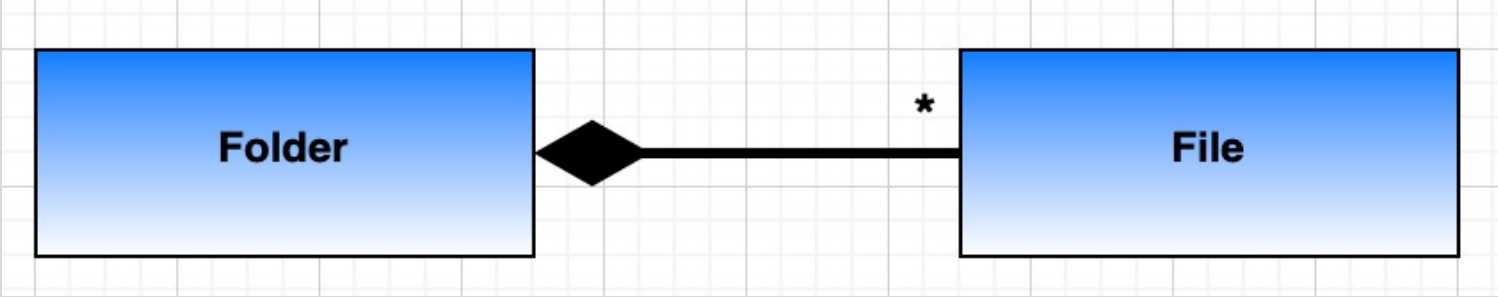
Aggregation

- Aggregation is a **strong** association to represent a "has-a" relationship
- In aggregation, if the container object is deleted, the contained objects can stay

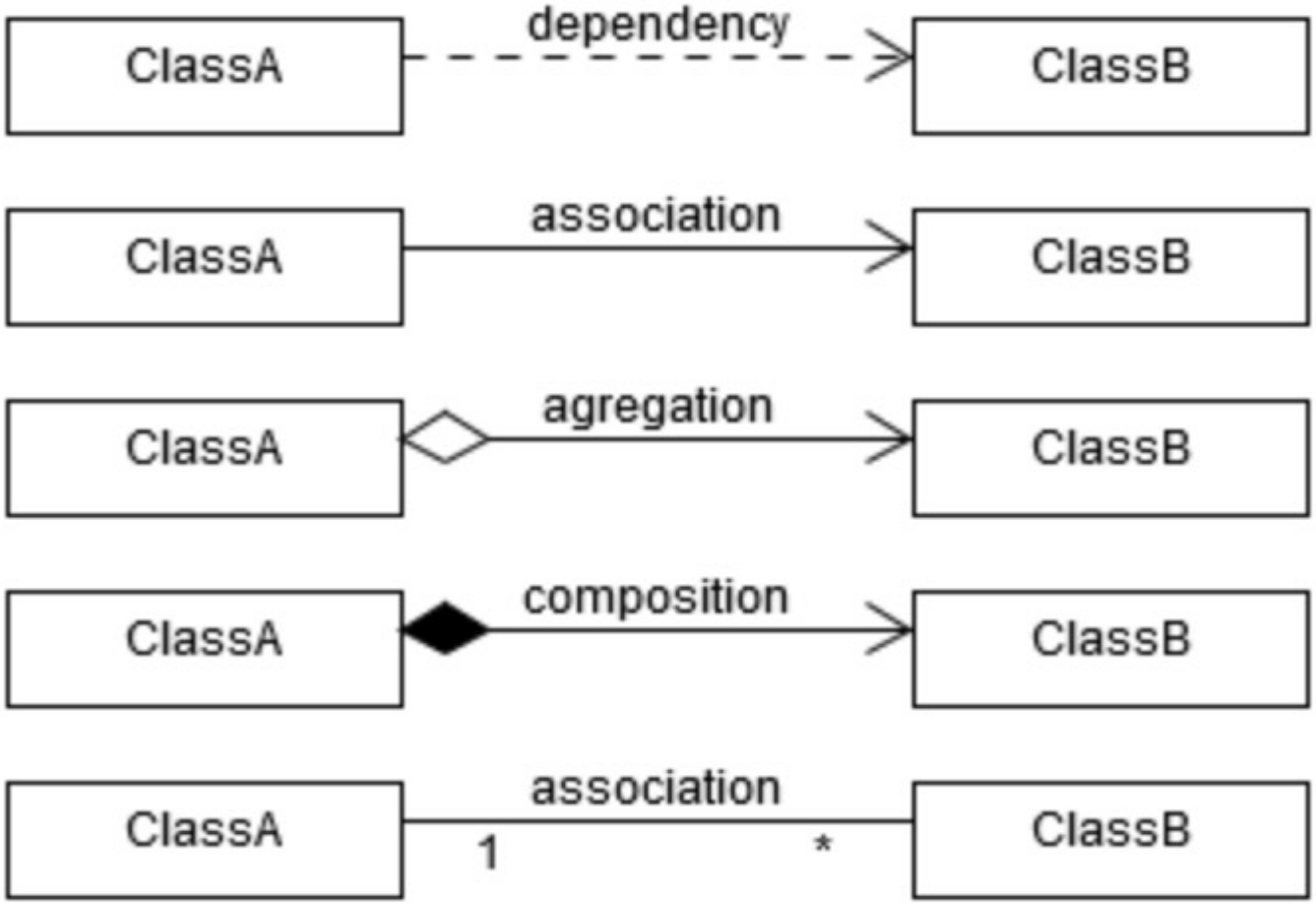


Composition

- Composition is a **strong** aggregation to represent a "contains" relationship
- In composition, if the container object is deleted, the contained objects must be deleted as well



Relationship Summary



Inheritance

RMIT UNIVERSITY

Method Overloading



Before looking at inheritance and the related idea of *overriding* a method it is useful to first consider (or revisit?) the simpler concept of method **overloading**

Java allows two or more methods to have the same *name* provided the methods vary in the type or number of arguments (NOTE: a different return type is not sufficient)

The actual method invoked is determined based on the passed argument type (at **compile time** from the ***declared*** type)

This process is known as method **overloading**

It is useful when we perform similar operations on different types and also applies to Constructors where it is particularly useful

Consider a Library Member class. We may want to `update()` or construct based on different types, especially when some details are optional

A good API example is the overloaded `println(…)` method in class `java.io.PrintStream`

Inheritance 101: The ‘supreme’ superclass Object



Every class (even one that does not use `extends` keyword) is a subclass of `java.lang.Object`

Therefore every time you instantiate a Java object inheritance is involved

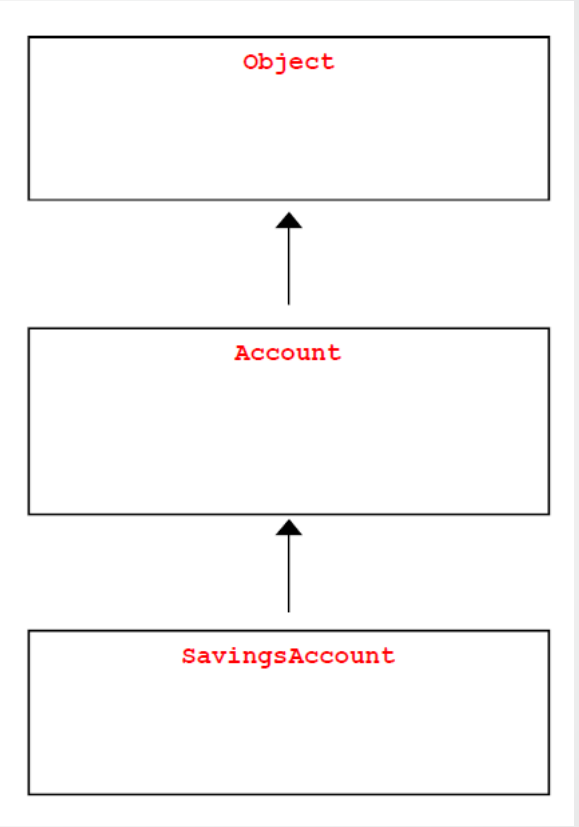
Why?

The `Object` class has a small number of methods that are useful for all classes

- `equals()`
- `toString()`
- etc.

Check the rest in the API docs!

Also provides a general type for variables/parameters but strong typing or generics (covered later) is better



Inheritance



In general, Inheritance and Polymorphism are strongly linked as we will see as we cover the topic further

However, when learning it is useful to treat them separately and start with Inheritance independently

Furthermore, it is possible to actually use inheritance on its own without any polymorphism so it is also reasonable from a practical perspective to start with this simple case

When using inheritance in this way the idea is to remove code duplication and promote code reusability by placing code that will be reused (and would otherwise have been duplicated across multiple classes) into a 'superclass'

This superclass is then extended using the Java `extends` keyword

```
class X extends Y { … }
```

Using general OO terminology the Java *extends* process is called *inheritance* since the *child/descendent/sub-* class inherits from the *parent/ancestor/super-* class (yep, lots of alternative terminology again!)

Object Oriented Relationships



Two commonly used clauses in class relationship

- **has-a**: Association/Composition/Aggregation relationship discussed in Lecture 1
- **is-a**: Inheritance relationship

A home **is-a** house that **has-a** family and a pet

If House, Family and Pet are existing classes then in Java we write

```
public class Home extends House
{
  private Family inhabitants;
  private Pet thePet;
}
```

Inheritance Overview



So we have identified that with inheritance a `Home` is a `House` but what does this actually mean in practice?

When we declare `class Home extends House` on the previous slide the `extends` (inheritance) keyword means that the `Home` class effectively contains all code from the `House` class that is not **private**

- i.e. it *inherits* all of the non-private attributes and methods of `House` even static methods
- These attributes can be modified and methods called as if they were part of the `House` class without having to rewrite/copy them
- This facilitates code *reuse!*

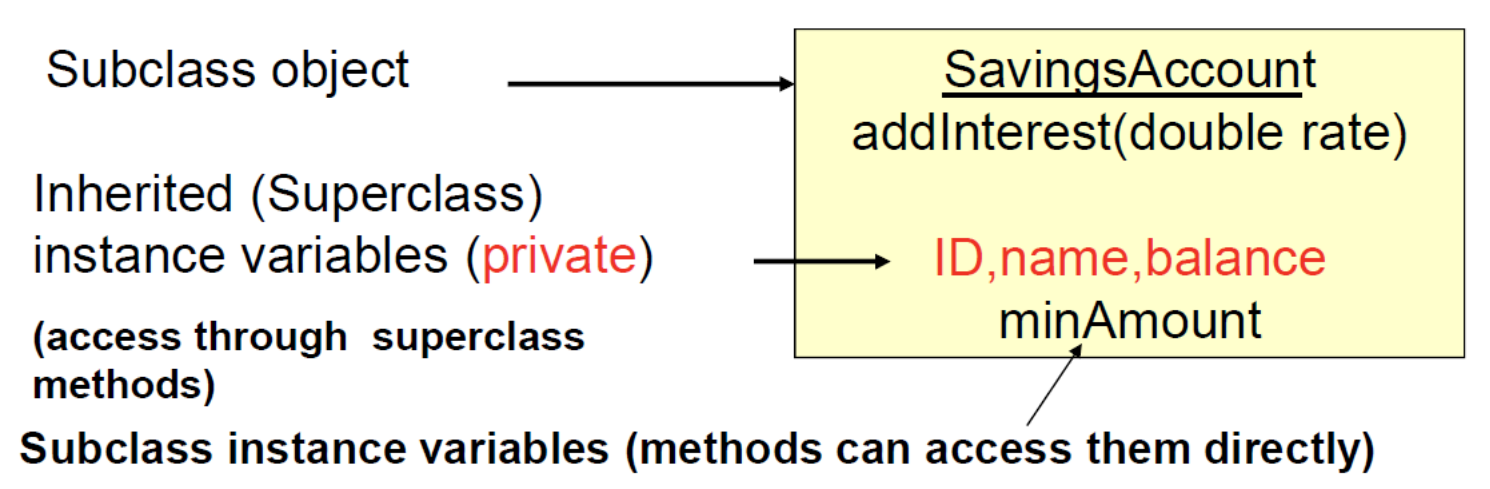
There are some extra rules here such as `final` methods but these will be covered later in the lecture/topic

Instance variable of superclasses

As stated on the previous slide any **non-private** code artefacts (attributes/methods) are automatically inherited by subclasses

But since attributes are usually **private** they cannot be accessed directly by subclasses

Hence the only way to change them is through superclass mutators/methods



Protected/Package Access



Protected and package (default) are alternatives to private and public

If an attribute/method is declared *protected* it can be accessed by methods of that *class,* its *subclasses (in ANY package)* and all *other classes* within the same *package*

Default/package is similar but subclasses must be in the same package

It can be argued that protected/package visibility strikes a balance between absolute protection and no protection at all

However, it breaks the *encapsulation* rule as the designer of the superclass has no control over the authors of the subclass

- Therefore the subclass can potentially break intended superclass behaviour

Therefore safer to use protected getters and setters instead to maintain encapsulation

- By ensuring that values stay within expected ranges, are modified in certain ways etc.

TIP: Where possible keep it simple and stick to public and private!

Inheritance: CreditAccount Example (1)



Consider a simple Account class

The class has:

- A *Name* (should be unique and descriptive)
- *Attributes* (also called fields/instance variables which are variables representing the **state** of the class)
- *Methods* (which are functions/procedures representing the **behaviour** of the class)
- *Relationships* to other classes represented as attributes (which may be arrays/collections for 'many' relationships)

NOTE: relationships are not explicit i.e. you need to understand the code to identify a relationship

Account
-acctNumber : int #balance : double
+withdraw(amount : double) : boolean +deposit(amount : double) +getAcctNumber() : integer +getBalance() : double

Inheritance: CreditAccount Example (2)



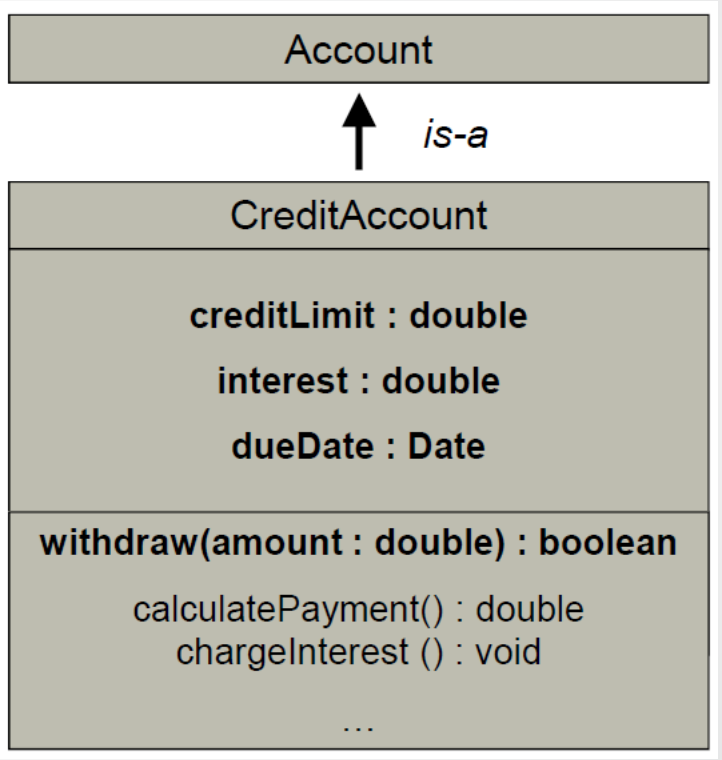
The bank introduces a new product, the credit account. This has a credit limit, an interest rate, a payment due date, and a payment amount outstanding

A customer may withdraw up to the credit limit

Every month, a credit charge based on the outstanding amount is be charged to the account

Inheritance promotes **reusability** (code reuse) and **extensibility** (extending or modifying existing code)

As stated previously every object in Java, either explicitly or implicitly, is a subclass of `java.lang.Object`.



Bank Account Example Summary



In summary, the `CreditAccount` class:

=> inherits non-private properties (attributes/methods) from the `Account` class

=> calls the superclass constructor to help create itself

=> overrides some methods with a new implementation

=> defines some new attributes and methods unique to `CreditAccount`

The super and this Keywords



A subclass may explicitly access a method or attribute in its superclass with the `super` keyword. When `super` is used to access the constructor of the superclass, it must be the first statement in the subclass's constructor (constructors are covered further later in the lecture)

The `this` keyword refers to the object through which the method or attribute is accessed (the current object.) In the absence of an object reference (or class name, for static items), this is the implied reference.

```
public CreditAccount(int num, double creditLimit, double interest)
{
  super(num, 0); // set the balance to 0 for new account
  this.creditLimit = creditLimit; // this resolves the
  this.interest = interest; // ambiguity
  balance = 0; // this is not necessary since no ambiguity
}
```

Using the 'this' Reference



this refers to methods and instance variables of current object

```
class SAccount extends Account
{
   public SAccount(String accountID, String
      accountName, double amount, double minAmount){
      super(accountID,accountName,amount);
      this.minAmount = minAmount;
   }
   . . .
   private double minAmount;
}
```

minAmount of current object (explicit)

Same name ?

Inherited Methods and Visibility



A subclass cannot override a method to become less visible. For example, the compiler will not allow this.

```
class Account
{
  public void withdraw(double amount) { . . . }
. . .
}
class SavingsAccount extends Account
{
  protected void withdraw(double amount) { . . . }
. . .
}
```

Question: Why is this restriction necessary?

Final methods and classes



REVISION: a final attribute/variable cannot be re-assigned

For methods, final prevents the method from being overridden by any inheriting class

Ensures that the behaviour of a method is retained during inheritance and cannot be overridden

For classes, final means they cannot be extended

When a class is declared final that class cannot be subclassed, and of course its methods cannot be overridden because the class itself cannot be extended

The class `java.lang.String` is defined:

```
public final class String
```

Therefore it cannot be extended (it is also an **immutable** class)

Inheritance and constructors



The behaviour of constructors in Java can be tricky if you do not understand how they work!

Classes do NOT inherit constructors

However, classes by default WILL call the superclass no argument constructor as their first line unless you explicitly call a different constructor with `super(...)` or `this(…)`

`super()` must eventually be called (or will get a recursive call error)

Therefore, by default constructors actually execute in reverse order from normal polymorphism in a class hierarchy, i.e., they are called recursively from the bottom until the top constructor executes (`java.lang.Object`) and then execute and return in a top down fashion

- This is different from method polymorphism where the VM starts at the most specialised subclass (at the bottom of the hierarchy) and works its way up until it finds a signature match

The best (most cohesive) approach is to use partial construction where each class in the hierarchy constructs only the parts specific to that class

Polymorphism

RMIT UNIVERSITY

Polymorphism



If we ask someone at home to feed the pets they know it means different things for different type of pets

Hence, we can consider the action **feed** to be polymorphic

In OOP, polymorphism promotes code reuse and extensibility by calling the method in a generic way

For example, we can deduct charges by calling an `applyFees()` method on all `Account` objects. But depending on the type of account the specific/different versions of operations such as `applyFees()` or `calculateInterest()` will be called for each account.

Polymorphism can also simplify class usage/understandability since the caller/client can treat related classes in the same way without any conditional code

Polymorphism and Dynamic Binding (1)



When the bank manager asks his/her staff to deduct $5.00 from every bank account, the staff know that different deduction methods will apply to different types of account. Hence the deduction action is polymorphic

When the method `withdraw()` is called on an `Account` object, depending on the actual type of that `Account` at the time, the correct version of `withdraw()` will be dynamically bound to that object and executed

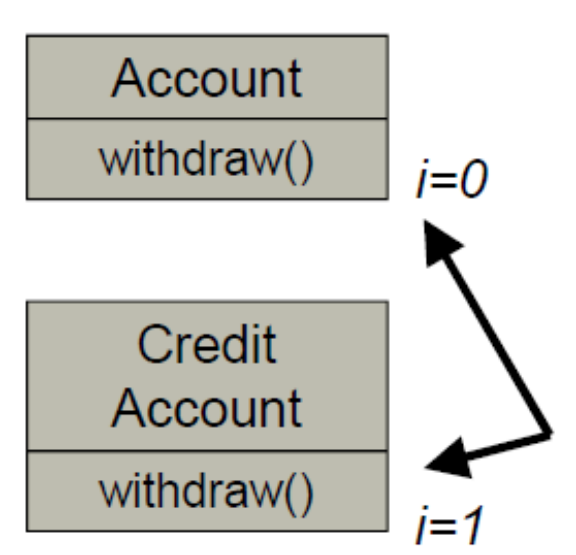
First, let's add several accounts of different types for a customer:

```
Customer john = new Customer("John", 123);
john.addAccount(new Account(1, 500));
john.addAccount(new CreditAccount(2, 0, 200));
```

Recall that the method `addAccount()` of `Customer` accepts an `Account` type as a parameter. Why can it also take in a `CreditAccount` type?

This is because `CreditAccount` "is-a" `Account`, due to inheritance

Polymorphism and Dynamic Binding (2)



Now, to withdraw $5.00 from all of John's accounts. Which version of `withdraw()` is executed when?

```
Customer john = new Customer("John", 123);
john.addAccount(new Account(1, 500));
john.addAccount(new CreditAccount(2, 0, 200));
for (int i = 0; i < 2; i++) {
  Account a = john.getAccount(i);
  a.withdraw(5);
}
```

We can put different types of `Account` in a common `Account` array, and withdrawal can be performed polymorphically on them.

`Account` is the base class of all account subclasses.

Polymorphism promotes code reuse by allowing methods to be called in a generic way.

Type Conversion



So, a `CreditAccount` can be treated as an `Account`. However, the reverse is not true; an `Account` may or may not be a `CreditAccount`

Can we convert an `Account` to a `CreditAccount`? Yes, by **type casting**. But this can be done only if the `Account` is actually a `CreditAccount`, otherwise a `ClassCastException` will be thrown

```
creditAc = (CreditAccount) ac;
```

A subclass reference is automatically a superclass reference, but the reverse is not true

It is possible to convert a superclass reference to a subclass reference if the former is actually referencing a subclass object, otherwise Java will throw a `ClassCastException`

Avoid casting even with *instance of* checks and maintain correct references and use polymorphism to handle type differentiation

Abstract Class

RMIT
UNIVERSITY

The abstract Modifier (1)



An abstract class

- Cannot be instantiated
- Is inherited and usually enhanced by subclasses
- May have zero or more abstract methods
- Cannot be instantiated using the `new` operator, but can define
  1) Constructors (to be invoked from subclass) and
  2) Methods to be invoked, implemented or overridden

The abstract Modifier (2)



An `abstract` method

- Method signature without implementation
- Cannot be contained in a non-abstract class
- If a subclass of an abstract superclass does not implement all the abstract methods, the subclass must be declared `abstract`
- Can exist in interfaces as well

Notes

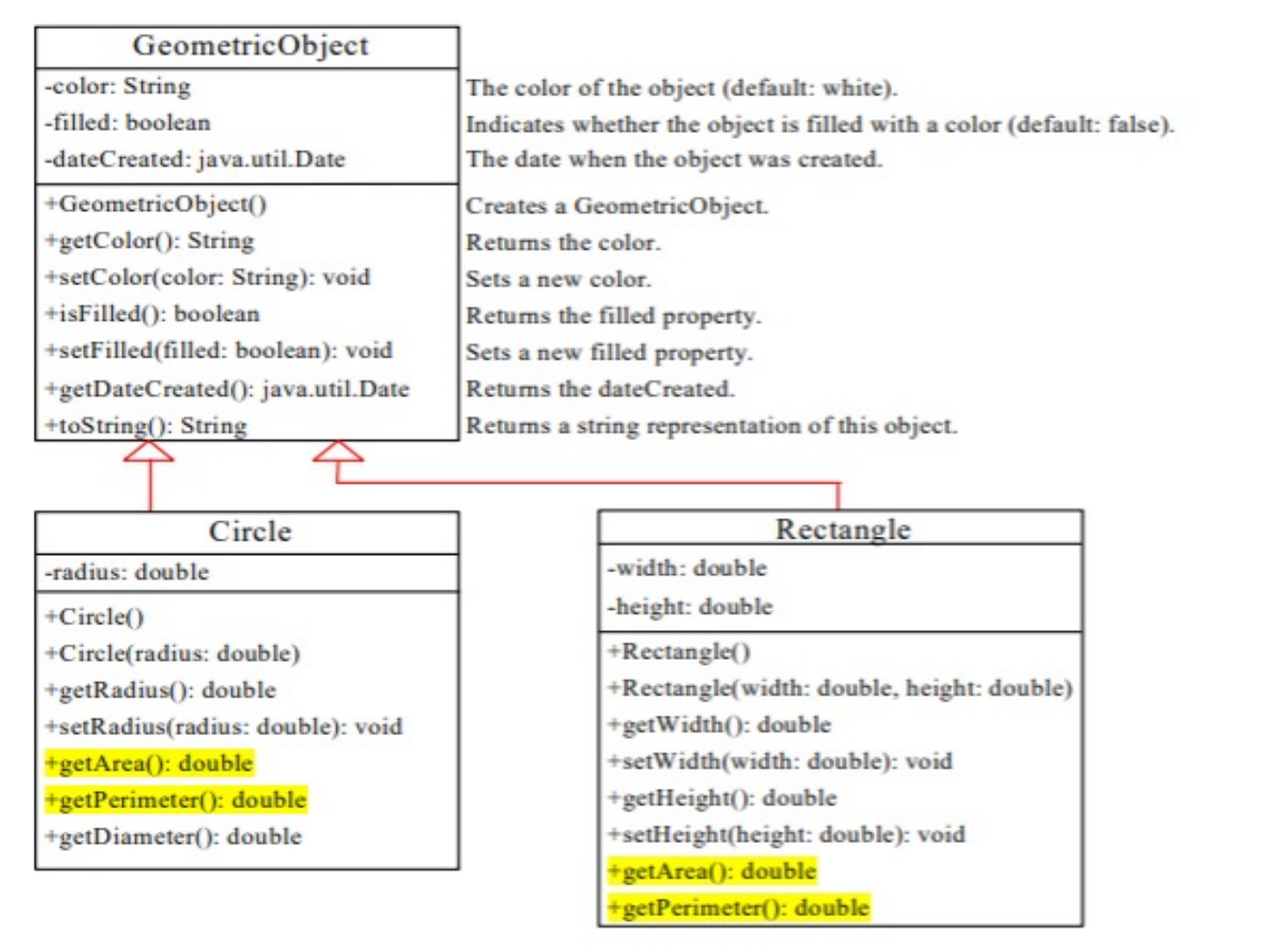


A subclass can be abstract even if its superclass is concrete. For example, the `java.lang.Object` class is concrete, but its subclasses, such as `GeometricObject`, may be abstract

You cannot create an instance from an abstract class using the `new` operator, but an abstract class can be used as a data type (declared/static type). Therefore, the following statement, which creates an array whose elements are of GeometricObject type, is correct:

```
GeometricObject[] geo = new GeometricObject[10];
geo[0] = new Circle(1.0);
```

Without abstract Class



Limitations

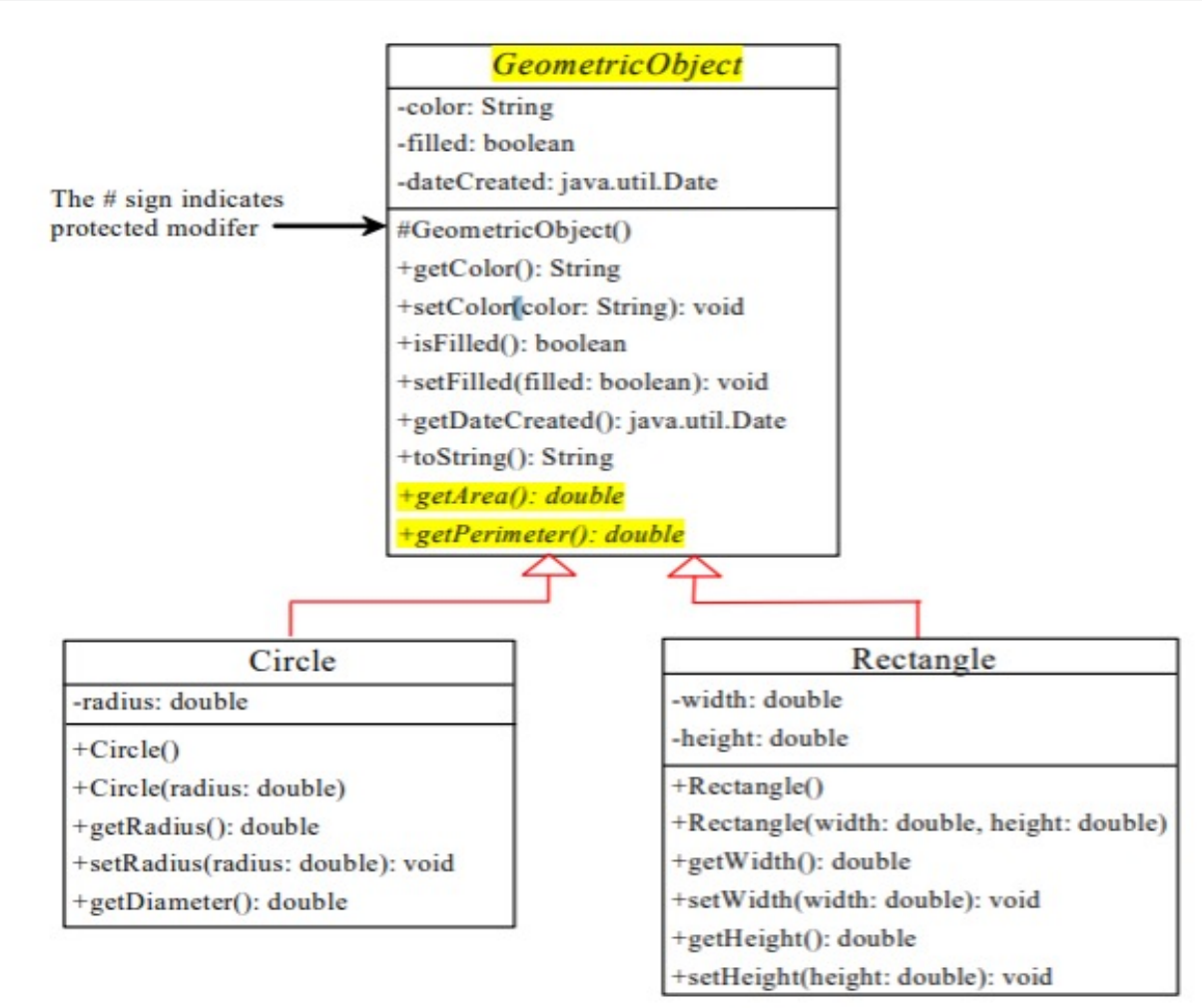


In this example we cannot call the common methods such as `getArea()` polymorphically

- Must treat each subtype differently and have separate collections/calls for each type
- Or need `if` statements and `instanceof` checks every time we want to make calls (which is rarely ever necessary!) if you group them
- Leads to a lot of duplicated code/redundancy

We could provide `getArea()` in `GeometricObject` but what would this mean and what would it return?

With abstract Class



About instanceof Operator



But what do I do when one of my subclasses has a specific method that doesn't exist in the rest of the hierarchy and I want to call it?

Option 1: Maintain a separate collection of that specialized type to call the specialized method

- avoids having to use `instanceof`
- but you do have to make sure objects get added to multiple collections when you create them (and remove later)
- Note: is only a duplication of the reference so is not expensive to do this

Option 2: instead of making the method(s) specific to the subclass create an interface and have the subclass implement that interface

- then use option 1 above but maintain a collection of the interface type instead
- Interfaces are covered in more detail from next slide

Interface

RMIT UNIVERSITY

Interfaces



An interface is a class-like construct that contains only constants and abstract methods. In many ways, an interface is similar to an abstract class, but an abstract class can contain variables and concrete methods as well as constants and abstract methods

To distinguish an interface from a class, Java uses the following syntax to declare an interface:

```
public interface InterfaceName {
  constant declarations;
  method signatures;
}
```

An interface is treated like a special class in Java. Each interface is compiled into a separate bytecode file, just like a regular class. Like an abstract class, you cannot create an instance from an interface using the new operator, but in most cases you can use an interface more or less the same way you use an abstract class. For example, you can use an interface as a data type for a variable or parameter type, as the result of casting (but avoid!), and so on

Interfaces: Non-Hierarchical Commonality

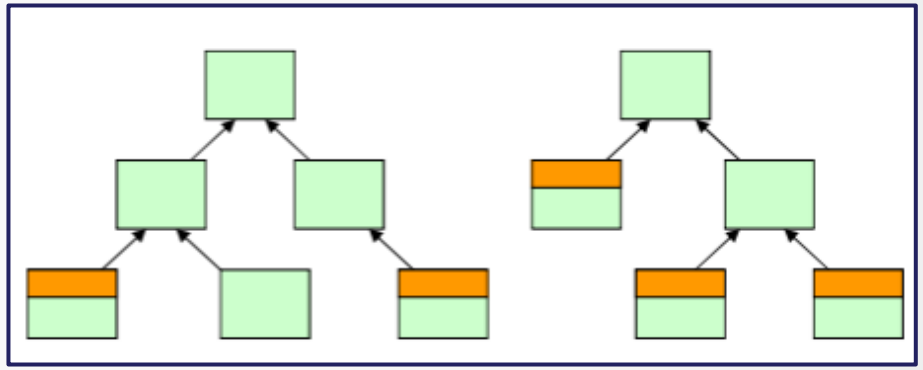


Consider a set of class hierarchies, some of whose objects can be printed out to a stream

We could **create an interface** defining how the "printable" functionality is to be implemented, and have these classes implement that interface

The benefit is that we can then use that interface **as a data** type, much like we can with superclasses and polymorphism

All `Printable` classes in the diagram can be referred to by their capability, even though they are completely unrelated in the hierarchy



Example



```
public interface Edible {
  /** Describe how to eat */
  public String howToEat();
}

class Animal {
}
```

```
class Chicken extends Animal implements
Edible {
  public String howToEat() {
    return "Fry it";
  }
}
```

```
abstract class Fruit implements Edible {}
```

```
class Apple extends Fruit {
  public String howToEat() {
    return "Make apple cider";
  }
}
```

```
class Orange extends Fruit {
      public String howToEat() {
            return "Make orange juice";
      }
}
```

Implements Multiple Interfaces



```
class Chicken extends Animal implements Edible, Comparable {
  int weight;
  public Chicken(int weight) {
    this.weight = weight;
  }

  public String howToEat() {
    return "Fry it";
  }

  public int compareTo(Object o) {
    return weight - ((Chicken)o).weight;
  }
}
```

Interfaces vs. Abstract Classes (1)



In an interface, the attributes must be constants (public final static) whereas an abstract class can have all types of data

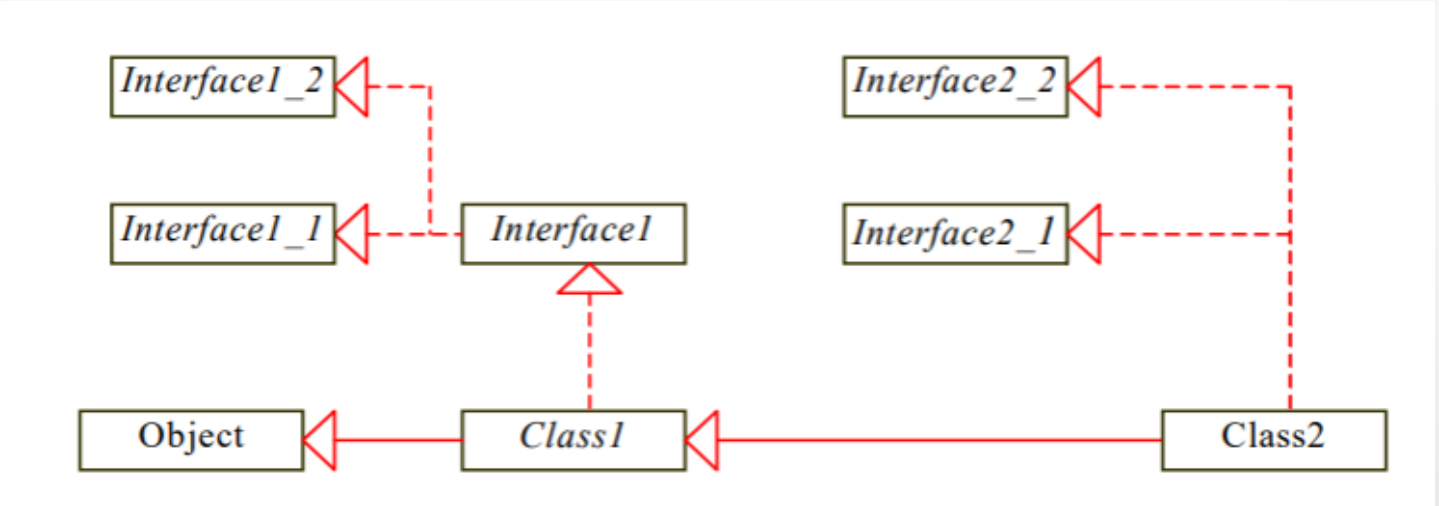
Each method in an interface has only a signature without implementation; an abstract class can have zero or more concrete methods with an implemented method body

	Variables	Constructors	Methods
Abstract class	No restrictions	Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated using the new operator	No restrictions
Interface	All variables must be public static final	No constructors. An interface cannot be instantiated using the new operator	All methods must be public abstract

Interfaces vs. Abstract Classes (2)

All classes share a single root, the **Object** class, but there is no single root for interfaces. Like a class, an interface also defines a type. A variable of an interface type can reference any instance of the class that implements the interface. If an interface extends an interface, this interface plays the same role as a superclass. You can use an interface as a data type for variables, parameters etc.

Suppose that c is an instance of Class2. c is also an instance of Object, Class1, Interface1, Interface1_1, Interface1_2, Interface2_1, and Interface2_2



Static vs. dynamic types and abstraction (1)



When we instantiate an object in Java we actually specify two types:

```
Fruit myFruit = new Apple();
```

The type on the left of the "=" is called the static or declared type (in the above example: `Fruit` interface)

- This type is used by the compiler to type check any references where myFruit is used (e.g. you cannot assign a `Fruit` to a `String`, you also cannot assign a `Fruit` to an `Apple` since not all `Fruit` are `Apple` (although all `Apple` instances are `Fruit`)
- Only methods or attributes that exist in the static/declared type can be accessed (regardless of the type on the right of "=")
- Try to use the most abstract type possible (highest in class hierarchy) to favour reuse and choose interfaces over classes where possible. This applies to **parameter types as well!**

Static vs. dynamic types and abstraction (2)



The type on the right hand side is the dynamic or runtime type. It is the actual type of the object when instantiated during runtime and determines which actual implementation of methods are used (e.g. `Apple.howToEat()` versus `Fruit.howToEat()`)

Having collections of interfaces or abstract types facilitates polymorphism by placing different dynamic/runtime types in the collection

Design with Abstract Classes and Interfaces (1)



Why abstract classes?

- We want a generalized base class that is incomplete (and thus cannot be accidentally instantiated) but want to provide default behavior (that may be extended or specialized later)
- We want the flexibility to add methods later without changing existing clients

Why interfaces?

- We want to treat different classes together in some specific way by means of a strict contractual interface (set of method signatures) but have no default behaviour and possibly no hierarchical relationship
- We have many such behaviours and therefore the need for multiple inheritance which is only supported at the interface level in Java

Design with Abstract Classes and Interfaces (2)



- Why interfaces?
  - We are prepared to “lock down” the interface as a specification since changing it has significant implications on clients
  - If we do add methods we usually create a new interface that extends the original but also keep the original for backwards compatibility
- Note that like regular inheritance, abstract classes and interfaces ensure a uniform and structured design throughout the class hierarchy. Abstract classes, inheritance and interfaces, together with polymorphism and dynamic binding, promote code reuse by allowing methods to be called in a generalized way. It also facilitates **maintainability by placing specialized code in a single place**

When to use an interface or a class?



In general for maximum extensibility it is preferable to use both interfaces and classes together rather than choose one or the other.

However:

- Can provide only an interface if there is some highly specialized functionality that must exist but is unlikely to have any commonality between classes (and thus must be implemented differently by all implementing classes)
- Can provide only classes where there is some very concrete functionality that is not expected to be changed or you need to ensure that a specific implementation is always used
- Can provide hierarchies of non-abstract classes where the class user can reasonably choose to instantiate classes at different levels in the hierarchy
- Can provide only classes or abstract classes if you need to add behaviors later without breaking existing clients

Designing for Inheritance



When designing for inheritance it is important to clearly **document** any self use in overridable methods (i.e., non-final, public, and protected)

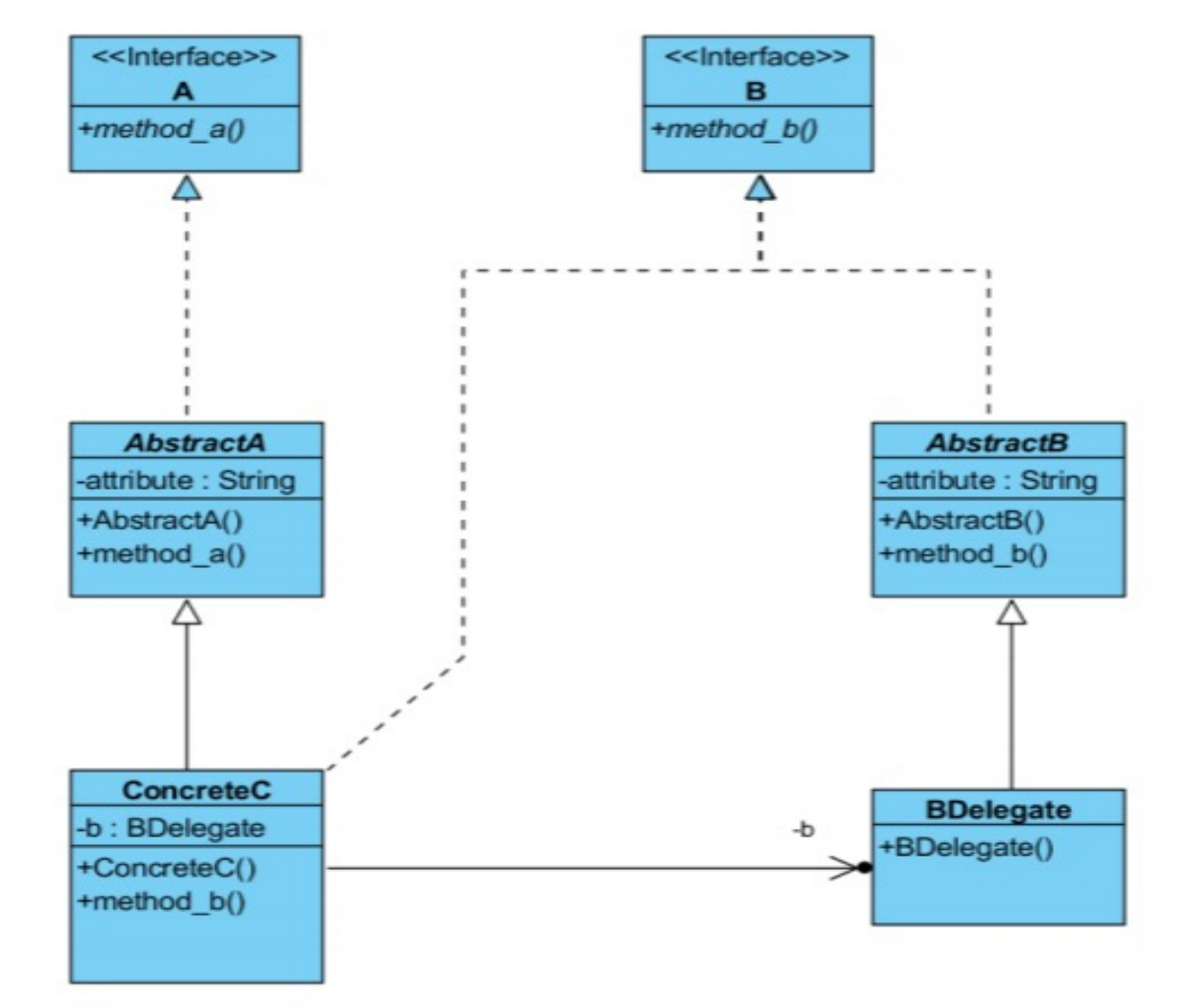
- This is important since otherwise a subclass may override a method and change expected behavior by not adhering to the required self-use, i.e. not calling other methods that the superclass expected to be called as part of its implementation

Judiciously expose “hooks” for extension by providing protected methods which are called at expected points of future difference

- Too many and you unnecessarily commit to implementation decisions
- Too few and the class lacks utility and cannot be meaningfully extended
- In rare cases you may also provide protected attributes although this is riskier and may lead to the subclass accidentally breaking superclass functionality

- **Superclass constructors should not invoke overridable methods since the method will run before the subclass constructor and may not have been properly initialized**

Delegation as an Alternative to Inheritance



Using Delegation to Simulate Multiple Inheritance (1)



Java has a limitation that only a single class can be extended (single implementation inheritance)

While you can implement multiple interfaces you may still have multiple default implementations (e.g. abstract classes) that you want to extend

The previous slide shows how this can be achieved using the following steps

- Assume two classes `AbstractA` and `AbstractB` that implement interface `A` and `B` respectively
- Class `ConcreteC` wants to extend both of these classes i.e. `ConcreteC` is both an `A` and a `B` (this is common e.g. a tutor could be both a student and a staff member)
- Class `ConcreteC` extends `AbstractA` (and therefore also inherits the A interface)
- Class `ConcreteC` implements B since it cannot extend both `AbstractA` and `AbstractB`
- Class `ConcreteC` instantiates `BDelegate` which extends `AbstractB` (therefore providing access to all of `AbstractB's` default behavior via interface B)
- Class `ConcreteC` implements interface `B` to pass all calls to `BDelegate`

Using Delegation to Simulate Multiple Inheritance (2)



- ConcreteC is as follows

```
// class C is an A (inherited through AbstractA)and a B
// alternatively we could extend AbstractB, and implement/delegate A
//
// Caspar's imaginary syntax
// public class ConcreteC extends AbstractA delegates interface B to BDelegate
public class ConcreteC extends AbstractA implements B
{
   // this attribute will implement the "B" functionality via the BDelegate class
   // and avoid us cutting and pasting (since we cannot extend two abstract classes)
   private B b=new BDelegate();

   public ConcreteC()
   {
      // this constructs the "A" portion via superclass AbstractA
      super();
   }

   // method_a() is inherited from superclass AbstractA

   // method_b() from interface B is delegated to attribute "b"
   // which avoids having to duplicate method body here
   public String method_b()
   {
      return b.method_b();
   }

   // forward other methods of B interface if it had more than one method
}
```

Using Delegation to Simulate Multiple Inheritance (3)



Note: this usage could be termed forwarding rather than delegation since in general delegation implies passing a self reference which is not the case in this example. However the functionality for `B` is delegated to a separate class and so delegation is an appropriate description in this case. This is also a good example of true **Composition** since the `BDelegate` is **part of** the `ConcreteC`

Java 8 Interface Default Methods



- Interfaces in Java 8+ add more features bringing them closer to true multiple implementation inheritance
- The only limitation now compared with abstract classes is that interfaces cannot have fields/attributes
- `default` methods in an interface are instance method implementations with a body that are automatically inherited when an interface is **implemented**
- Can be called directly from a subclass to resolve conflict: `A.super.methodName();`
- Allows pseudo multiple implementation inheritance i.e. class inherits method implementation (not just signatures) from multiple classes

```
public interface Vehicle {

    String getBrand();

    String speedUp();

    String slowDown();

    default String turnAlarmOn() {
        return "Turning the vehicle alarm on.";
    }

    default String turnAlarmOff() {
        return "Turning the vehicle alarm off.";
    }
}
```

https://www.baeldung.com/java-static-default-methods

Thank you

COSC2440 teaching team

RMIT UNIVERSITY